CS 225

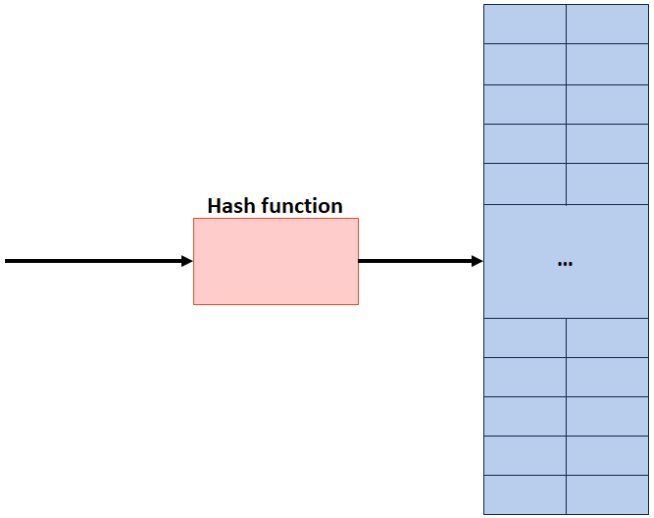
#25: Hashing

April 15-17, 2026 V. Kindratenko

Goals for Understanding Hashing:

1. We will define a **keyspace**, a (mathematical) description of the keys for a set of data.

2. We will define a function used to map the **keyspace** into a small set of integers.



Dictionary ADT in Client Code	
1	`Dictionary<KeyType, ValueType> d;`
2	`d[k] = v;`

A hash table consists of three things:

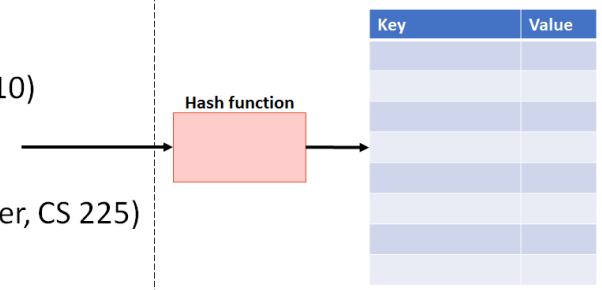
1.

2.

3.

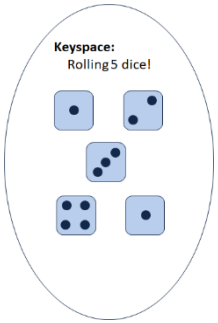
A Perfect Hash Function

(Angrave, CS 241)
(Beckman, CS 421)
(Cunningham, CS 210)
(Davis, CS 101)
(Evans, CS 126)
(Fagen-Ulmschneider, CS 225)
(Gunter, CS 422)
(Herman, CS 233)



...characteristics of this function?

A Second Hash Function



0	
1	
2	
3	
4	
5	
6	
7	
8	
9	
10	
11	
12	
13	
14	
15	

...characteristics of this function?

All hash functions will consist of two parts:

- A **hash**:
- A **compression**:

Characteristics of a good hash function:

1. Computation Time:

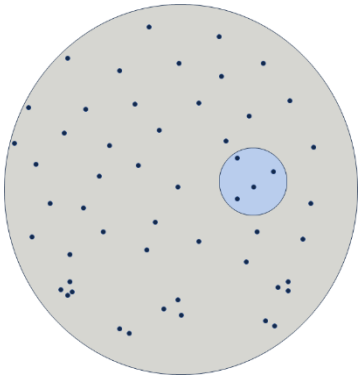
2. Deterministic:

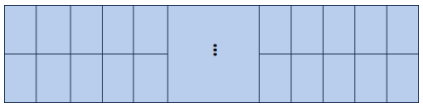
3. SUHA:

Towards a general-purpose hashing function:
It is <u>easy to create</u> a general-purpose hashing function when the keyspace is proportional to the table size:

- **Ex:** Professors at CS@Illinois
- **Ex:** Anything you can reason about every possible value

It is difficult to create a general-purpose hashing function when the keyspace is large:





Ex: Hashing a string of 8 characters is considered easy, 40 is not.

My 40-character strategy:

	Alice in Wonderland, Page 1
1	Alice was beginning to get very tired of
2	sitting by her sister on the bank, and
3	of having nothing to do: once or twice s
4	he had peeped into the book her sister w
5	as reading, but it had no pictures or co
6	nversations in it, 'and what is the use
7	of a book,' thought Alice 'without pictu
8	res or conversations?' So she was consi
9	dering in her own mind (as well as she c
10	ould, for the hot day made her feel very
11	sleepy and stupid), whether the pleasur
12	e of making a daisy-chain would be worth
13	the trouble of getting up and picking t
14	he daisies, when suddenly a White Rabbit
15	with pink eyes ran close by her. There
16	was nothing so very remarkable in that;
17	nor did Alice think it so very much out
18	of the way to hear the Rabbit say to it
19	self, 'Oh dear! Oh dear! I shall be late
20	!' (when she thought it over afterwards,
21	it occurred to her that she ought to ha

...characteristics of this function?

Reflections on Hashing
In CS 225, we are starting the study of general-purpose hash functions. There are many other types of hashes for specific uses:

- **Common**: Cryptographic hash functions

Even if we build a good hash function, it is not perfect. What happens when the function isn't always a bijection?

CS 225

#26: Hashing, Part 2

April 15-17, 2026 V. Kindratenko

Dictionary ADT in Client Code	
1	`Dictionary<KeyType, ValueType> d;`
2	`d[k] = v;`

A hash table consists of three things:

1.

2.

3.

Collision Handling Strategy #1: Separate Chaining

Example: **S = { 16, 8, 4, 13, 29, 11, 22 }, |S| = n**
h(k) = k % 7, |Array| = N

[0]	
[1]	
[2]	
[3]	
[4]	
[5]	
[6]	
[7]	

Load Factor:

Running time of Separate Chaining:

	Worst Case	SUHA
Insert		
Remove/Find		

Collision Handling Strategy #2: Probe-based Hashing

Example: **S = { 16, 8, 4, 13, 29, 11, 22 }, |S| = n**
h(k) = k % 7, |Array| = N

[0]	
[1]	
[2]	
[3]	
[4]	
[5]	
[6]	
[7]	

Linear Probing:
Try h(k) = (k + 0) % 7, if full...
Try h(k) = (k + 1) % 7, if full...
Try h(k) = (k + 2) % 7, if full...
...

Linear Probing leads to Primary Clustering

Description:

Remedy:

Double Hashing:

Example: **S = { 16, 8, 4, 13, 29, 11, 22 }, |S| = n**
$h_1(k) = k \% 7$, $h_2(k) = (5 - k) \% 5$, |Array| = N

[0]	
[1]	
[2]	
[3]	
[4]	
[5]	
[6]	
[7]	

Double Hashing:
Try $h(k) = (k + + 0*h_2(k)) \% 7$, if full…
Try $h(k) = (k + + 1*h_2(k)) \% 7$, if full…
Try $h(k) = (k + + 2*h_2(k)) \% 7$, if full…
…

$h(k, i) = (h_1(k) + i*h_2(k)) \% 7$

Running Time:
Linear Probing:
- Successful: **$\frac{1}{2}(1 + 1/(1-\alpha))$**
- Unsuccessful: **$\frac{1}{2}(1 + 1/(1-\alpha))^2$**

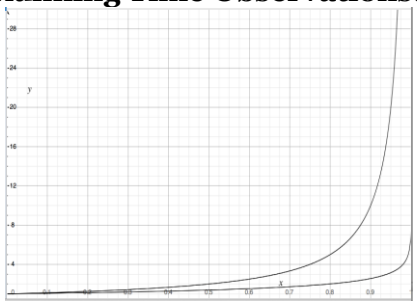
Double Hashing:
- Successful: **$1/\alpha * \ln(1/(1-\alpha))$**
- Unsuccessful: **$1/(1-\alpha)$**

Separate Chaining:
- Successful: **$1 + \alpha/2$**
- Unsuccessful: **$1 + \alpha$**

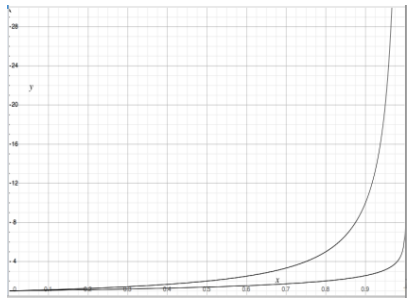
Running Time Observations:
1. As **α** increases:

2. If **α** is held constant:

Running Time Observations:



Linear Probing:
Successful: **$\frac{1}{2}(1 + 1/(1-\alpha))$**
Unsuccessful: **$\frac{1}{2}(1 + 1/(1-\alpha))^2$**



Double Hashing:
Successful: **$1/\alpha * \ln(1/(1-\alpha))$**
Unsuccessful: **$1/(1-\alpha)$**

ReHashing:
What happens when the array fills?

Better question and algorithm:

Which collision resolution strategy is better?
- Big Records:

- Structure Speed:

What structure do hash tables replace?

What constraint exists on hashing that doesn't exist with BSTs?

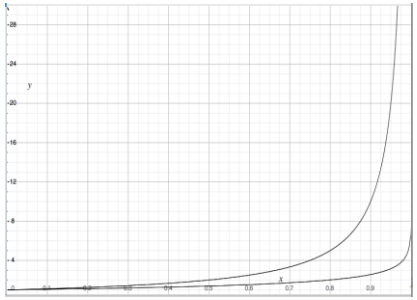
Why talk about BSTs at all?

CS 225

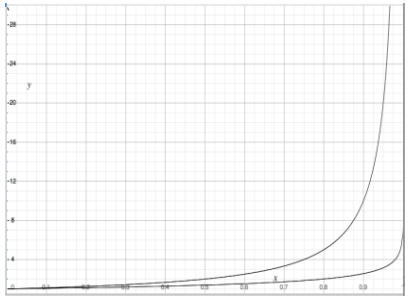
#27: Hashing, Part 3; Priority Queue

April 15-17, 2026 V. Kindratenko

Running Time Observations:



Linear Probing:
Successful: $\frac{1}{2}(1 + 1/(1-\alpha))$
Unsuccessful: $\frac{1}{2}(1 + 1/(1-\alpha))^2$



Double Hashing:
Successful: $1/\alpha * \ln(1/(1-\alpha))$
Unsuccessful: $1/(1-\alpha)$

ReHashing:
What happens when the array fills?

Better question:

Algorithm:

	Hash Table		AVL	List
	Amortized	Worst Case		
Find				
Insert				
Storage Space				

Which collision resolution strategy is better?

- Big Records:
- Structure Speed:

What structure do hash tables replace?

What constraint exists on hashing that doesn't exist with BSTs?

Why talk about BSTs at all?

A Secret, Mystery Data Structure:

ADT:
insert

remove

isEmpty

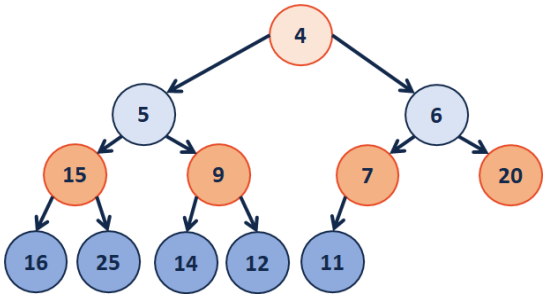
Priority Queue Implementations

insert	removeMin	Implementation
O(n)	O(n)	Unsorted Array
O(1)	O(n)	Unsorted List
O(lg(n))	O(1)	Sorted Array
O(lg(n))	O(1)	Sorted List

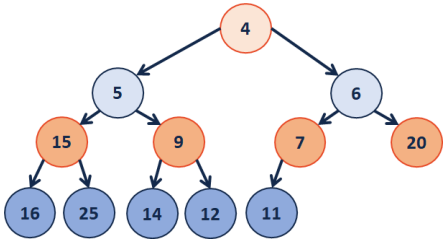
...what errors exist in this table?

Which algorithm would we use?

A New Tree-like Structure:



Implementing a (min)Heap as an Array



4	5	6	15	9	7	20	16	25	14	12	11			

CS 225 – Things To Be Doing:

1. MP5 **Extra credit** due next Monday
2. lab_btree due this Sunday